

IMPLEMENTATION DOCUMENTATION

[Draw your reader in with an engaging abstract. It is typically a short summary of the document. When you're ready to add your content, just click here and start typing.]

1.1

Ntlahla
Nongciki(226083470)
Senamile
Mhlaba(229922171)

Table of Contents

Implementation Document.....	3
Introduction	3
1.1 Choice of Tools	3
1.1.1 Microsoft Visual Studio 22	3
1.1.2 ASP.NET Core Web API	3
1.1.3 C#	3
1.1.4 Entity Framework Core	4
1.1.5 Microsoft SQL Server Express	4
1.1.6 SQL Server Management Studio.....	4
1.1.7 React	4
1.1.8 Google AI Studio and Lovable	4
1.1.9 Swagger	4
1.1.10 Git.....	4
1.1.11 GitHub	4
1.1.12 Microsoft Edge.....	5
1.1.13 Microsoft Word	5
1.1.14 WhatsApp	5
1.1.15 ChatGPT.....	5
1.1.16 DeepSeek.....	5
1.1.19 Microsoft Learn and Online Technical Documentation	5
1.1.20 Previous Academic Material and GitHub Projects	5
1.1.21 Microsoft Excel	5
1.2 Extracts of Complex Code.....	6
1.2.1 Booking Conflict Validation.....	6
1.2.2 Booking Approval and Rejection.....	7
1.2.3 Notification Retrieval and User-Specific Access	8
1.2.4 Audit Logging.....	9
1.3 Source Code References	10
1.3.1 Role-Based Authorisation and JWT Authentication	10
1.3.2 Booking Conflict Validation.....	11
1.3.3 Conference Booking and Approval Processes	11

1.3.4 User Notifications	11
1.3.5 Audit Logging	12
1.3.6 Summary of Adapted External Sources	12
1.4 Problems Encountered	13
1.4.1 Developing the Project Idea and Requirements	13
1.4.2 Designing the System Before Development	13
1.4.3 Creating the Database Structure	13
1.4.4 Complexity of the Bookings Controller	14
1.4.5 Authentication and Authorisation	15
1.4.6 Booking Availability and Conflicting Bookings	15
1.4.7 Managing Booking Statuses	15
1.4.8 Frontend and Backend Communication	16
1.4.9 Testing Role-Specific Functionality	16
1.4.10 Coordinating Work Between Team Member	16
1.4.11 Limited Development Time and Laboratory Access	16
1.4.12 Slow Development Computers	17
1.4.13 Managing the Project Timeline	17
1.5 Implementation Summary	17
References	18



Implementation Document

Introduction

4.1

The implementation phase involved turning the requirements and technical design of the DIBA Bookings system into a working web application. The system was developed to manage venues, events, bookings, users, notifications, payments and administrative activities at the Nelson Mandela University Conference Centre. The system has three main user roles: Event Organisers, Conference Centre Staff and System Administrators. Each role has different permissions based on its responsibilities.

The application consists of a React frontend, an ASP.NET Core Web API backend and a Microsoft SQL Server database. The backend manages the system rules, authentication, authorisation and communication with the database. This document describes the tools used, complex code developed, external sources used, and problems encountered during implementation and how they were solved.

1.1 Choice of Tools

The tools used to develop the DIBA Bookings system were selected based on the project requirements, the system design and the different stages of development. They supported programming, database development, interface design, testing, version control, communication, documentation and technical research.

1.1.1 Microsoft Visual Studio 2022

Microsoft Visual Studio was used as the main development environment for the backend. It was used to create the ASP.NET Core Web API, write C# code, manage the project and debug errors.

It was selected because it supports C#, ASP.NET Core and Entity Framework Core, which were required for the backend.

1.1.2 ASP.NET Core Web API

ASP.NET Core Web API was used to develop the backend of the system. It connects the React frontend to the database and handles functions such as authentication, authorisation, validation, bookings, venues, events, notifications and payments. The API also allowed the backend to be tested separately from the frontend.

1.1.3 C#

C# was used as the main programming language for the backend. It was used to create controllers, entities, data transfer objects, authentication logic and other backend functionality. C# also allowed the main parts of the system, such as users, venues, events and bookings, to be represented as classes.

1.1.4 Entity Framework Core

Entity Framework Core was used to connect the C# backend to the database. It allowed database tables and relationships to be represented using C# classes. It was used for tasks such as creating, retrieving and updating records, checking bookings and managing relationships between entities.

1.1.5 Microsoft SQL Server Express

Microsoft SQL Server was used as the database management system. It stores information about users, roles, venues, venue features, events, bookings, notifications, payments and audit logs. Primary keys and foreign keys were used to connect related records. SQL Server was selected because it was part of the project requirements and works well with ASP.NET Core and Entity Framework Core.

1.1.6 SQL Server Management Studio

SQL Server Management Studio (SSMS) was used to manage and inspect the database during development. It was used to view tables and records, run SQL queries and check whether information was being correctly saved by the application.

1.1.7 React

React was used to develop the frontend of the system. It was used to create the pages, forms, navigation, dashboards and other parts of the user interface. React communicates with the ASP.NET Core API to send and receive information.

1.1.8 Google AI Studio and Lovable

Google AI Studio and Lovable were used to plan and design the user interface before and during development. It helped the team plan the layout, navigation, colours, forms and general appearance of the system before implementing the designs in React.

1.1.9 Swagger

Swagger was used to document and test the backend API. It allowed the team to view the API endpoints and send requests directly to the backend. This helped test features before connecting them to the React frontend.

1.1.10 Git

Git was used for version control. It allowed changes to the source code to be tracked and previous versions to be reviewed when problems occurred.

1.1.11 GitHub

GitHub was used to store and manage the project's source code repositories. It provided a central location for the code and supported collaboration between team members. GitHub was also useful for keeping a remote copy of the project.

1.1.12 Microsoft Edge

Microsoft Edge was used to run and test the React frontend. It was used to inspect the frontend, identify console errors and check network requests between the frontend and backend.

1.1.13 Microsoft Word

Microsoft Word was used to prepare the project documentation and reports. It was used for writing, formatting and organising the required project documents.

1.1.14 WhatsApp

WhatsApp was used for communication and project coordination. It was used to discuss tasks, share updates, arrange meetings and communicate about development problems.

1.1.15 ChatGPT

ChatGPT was used as a supporting development and research tool. It helped the team understand technical concepts, investigate errors, discuss possible solutions and improve explanations. Suggestions from the tool were reviewed and adapted by the team before being used.

1.1.16 DeepSeek

DeepSeek was used as an additional AI-assisted development tool. It was used to help understand programming problems, compare possible solutions and assist with debugging. The suggestions provided were reviewed and tested before being used in the project.

1.1.19 Microsoft Learn and Online Technical Documentation

Microsoft Learn and other official technical documentation were used as reference material during development. They were used to understand technologies such as ASP.NET Core, Entity Framework Core, C#, authentication and other Microsoft technologies.

1.1.20 Previous Academic Material and GitHub Projects

Previous academic material, lecture content and relevant GitHub projects were used as technical reference material. These resources helped the team understand implementation approaches and programming concepts. Any externally sourced code or material used in the system is discussed separately in Section 1.3.

1.1.21 Microsoft Excel

Microsoft Excel and other project coordination methods were used to organise the development work. The team used these methods to track tasks, coordinate

responsibilities, monitor progress and work towards project deadlines. These methods helped the team divide the project into smaller tasks and keep track of work that still needed to be completed.

1.2 Extracts of Complex Code

The DIBA Bookings system contains several sections of code that required more development work than simple database operations. The selected extracts below focus on code that implements important business rules and system processes. The extracts show how the requirements of the DIBA Bookings system were implemented in the final application.

1.2.1 Booking Conflict Validation

Booking conflict validation was an important part of the booking process. The system must prevent two bookings from using the same venue at overlapping times. The following code checks whether the selected venue already has a pending or approved booking during the requested time:

```
var hasConflict = await _dbContext.Bookings
    .AnyAsync(booking =>
        booking.VenueId ==
        createBookingDto.VenueId &&

        booking.StartDateTime <
        createBookingDto.EndDateTime &&

        booking.EndDateTime >
        createBookingDto.StartDateTime &&

        booking.BookingStatus != null &&
        (
            booking.BookingStatus.StatusName == "Pending" ||
            booking.BookingStatus.StatusName == "Approved"
        ));

if (hasConflict)
{
    return Conflict(
        "The selected venue is already booked or awaiting approval
for the requested time.");
}
```

The code checks the venue, start time, end time and booking status. If an existing booking overlaps with the requested time, the system does not allow the new booking to continue. A *409 Conflict* response is returned to the frontend when a conflict is found. After the checks have passed, the booking is created with a Pending status:

```
var booking = new Booking
{
    BookingId = Guid.NewGuid(),
```



```

        BookingDate = DateTime.UtcNow,
        StartDateTime =
            createBookingDto.StartDateTime,
        EndDateTime =
            createBookingDto.EndDateTime,
        SpecialRequirements =
            createBookingDto.SpecialRequirements,
        AdminNotes = null,
        UserId = userId,
        EventId = createBookingDto.EventId,
        VenueId = createBookingDto.VenueId,
        BookingStatusId =
            pendingStatus.BookingStatusId
    };

    _dbContext.Bookings.Add(booking);

    await _dbContext.SaveChangesAsync();

```

This makes the booking process more than a simple database insert because the system checks whether the booking is allowed before saving it.

1.2.2 Booking Approval and Rejection

Booking approval and rejection form part of the administrative workflow. A booking must first be in a Pending state before it can be approved. The following code checks the current booking status:

```

    if (!currentStatus.StatusName.Equals(
        "Pending",
        StringComparison.OrdinalIgnoreCase))
    {
        return BadRequest(
            "Only pending bookings can be updated.");
    }

    booking.BookingStatusId =
        approvedStatus.BookingStatusId;

```

When a booking is approved, the system also records the action and creates a notification for the Event Organiser.

```

    var auditLog = new AuditLog
    {
        AuditLogId = Guid.NewGuid(),
        Action = "Booking Approved",
        LogDescription =
            $"Booking {booking.BookingId} was approved.",
        Timestamp = DateTime.UtcNow,
        UserId = userId
    };

    _dbContext.AuditLogs.Add(auditLog);

    var notification = new Notification
    {

```




```

        NotificationId = Guid.NewGuid(),
        NotificationType = "Booking Approved",
        Message =
            "Your venue booking has been approved.",
        DateCreated = DateTime.UtcNow,
        IsRead = false,
        UserId = booking.UserId,
        BookingId = booking.BookingId
    };

    _dbContext.Notifications.Add(notification);

    await _dbContext.SaveChangesAsync();

```

This means that one administrative action updates the booking, records the action and informs the user.

The rejection process also checks that a reason has been provided:

```

if (string.IsNullOrEmpty(
    rejectBookingDto.Reason))
{
    return BadRequest(
        "A rejection reason is required.");
}

booking.AdminNotes =
    rejectBookingDto.Reason;

booking.BookingStatusId =

    rejectedStatus.BookingStatusId;

```

The rejection reason is stored with the booking and can be included in the notification sent to the Event Organiser.

1.2.3 Notification Retrieval and User-Specific Access

The notification system was implemented so that users only receive notifications belonging to their own account.

The backend first obtains the ID of the logged-in user:

```

if (!UserHelper.TryGetUserId(
    User,
    out Guid userId))
{
    return Unauthorized(
        "User ID could not be determined.");
}

```

The system then uses this ID when retrieving notifications:

```

if (!UserHelper.TryGetUserId(User, out Guid userId))

```

```

{
    return Unauthorized("User ID could not be determined.");
}

var notifications = await _dbContext.Notifications
    .Where(notification => notification.UserId == userId)

    .OrderByDescending(notification => notification.DateCreated)

    .Select(notification => new NotificationResponseDto
    {
        NotificationId = notification.NotificationId,
        NotificationType = notification.NotificationType,
        Message = notification.Message,
        DateCreated = notification.DateCreated,
        IsRead = notification.IsRead,
        UserId = notification.UserId,
        BookingId = notification.BookingId
    })

    .ToListAsync();

```

The Where() condition makes sure that only notifications belonging to the logged-in user are returned. The OrderByDescending() method places the newest notifications first.

The system can also mark unread notifications as read:

```

var notifications = await _dbContext.Notifications
    .Where(notification =>
        notification.UserId == userId &&
        !notification.IsRead)
    .ToListAsync();

foreach (var notification in notifications)
{
    notification.IsRead = true;
}

await _dbContext.SaveChangesAsync();

```

This allows the system to keep track of which notifications have already been viewed.

1.2.4 Audit Logging

Audit logging was implemented to keep a record of important actions performed in the system. An audit record contains information such as the action, description, time and user who performed the action.

The following code retrieves audit records and the user connected to each record:

```

var auditLogs = await _dbContext.AuditLogs
    .Include(auditLog =>
        auditLog.User)
    .OrderByDescending(auditLog =>
        auditLog.Timestamp)

```

```

.Select(auditLog =>
    new AuditLogResponseDto
    {
        AuditLogId =
            auditLog.AuditLogId,

        Action =
            auditLog.Action,

        LogDescription =
            auditLog.LogDescription,

        Timestamp =
            auditLog.Timestamp,

        UserId =
            auditLog.UserId,

        UserName =
            auditLog.User == null
            ? string.Empty
            : auditLog.User.FirstName +
              " " +
              auditLog.User.LastName
    })

.ToListAsync();

```

The code retrieves the audit records, includes the related user, places the newest records first and prepares the information needed by the frontend.

Audit records are also created when important actions such as booking approval, rejection and cancellation take place. This allows administrators to see what action was performed and which user performed it.

1.3 Source Code References

External sources were used during development to understand common programming solutions and implementation approaches. These sources included GitHub projects, Microsoft Learn and Stack Overflow. The sources were used as technical references and, where appropriate, ideas or code were adapted to meet the requirements of the DIBA Bookings system. The final implementation was changed to fit the system's database structure, user roles and business rules.

1.3.1 Role-Based Authorisation and JWT Authentication

Description

External examples were used to understand how JWT authentication and role-based authorisation can be implemented in an ASP.NET Core Web API. The examples helped with understanding how users can be authenticated and how access can be restricted according to their roles. The ASP.NET Core documentation also explains how the [Authorize] attribute can be used to restrict access to specific roles. (olaideogunbunmi, 2026); (Microsoft, 2026)



Usage in DIBA Bookings

This approach was adapted for the DIBA Bookings backend. The system uses different roles, including Administrator, Conference Centre Staff and Event Organiser. Role restrictions are used on protected controllers and actions, such as administrative functions and booking management.

1.3.2 Booking Conflict Validation

Description

A meeting-room reservation project was reviewed to understand how a booking system can prevent two users from reserving the same room for overlapping times. The project uses validation to prevent overlapping reservations (ouardini14, 2022).

Usage in DIBA Bookings

The approach was adapted for the DIBA Bookings system. Before a booking is created, the backend checks whether another booking exists for the selected venue and time period. Pending and Approved bookings are considered when checking for conflicts. If a conflict is found, the system stops the booking from being created.

1.3.3 Conference Booking and Approval Processes

Description

A conference booking API was reviewed to understand how a backend system can manage conference rooms, reservations and user access. The project uses ASP.NET Core and includes booking management and role-based access (JedAngelo, 2024). Also was reviewed to understand how bookings, events, notifications and administrative actions can work together in one system.

Usage in DIBA Bookings

These ideas were adapted for the DIBA booking process. An Event Organiser creates a booking, after which Conference Centre Staff can approve or reject it. The system also requires a rejection reason when a booking is rejected. When a booking is approved or rejected, the system creates a notification for the Event Organiser and records the action in the audit log.

1.3.4 User Notifications

Description

An ASP.NET Core notification service was reviewed to understand how notifications can be stored and provided to individual users. The project includes single-user notifications and read/unread tracking. (Hisham, 2026)

A personal finance dashboard project was also reviewed because it includes a notification service with user-based notification retrieval and a mark-as-read function (Iriarte, 2026).

Usage in DIBA Bookings

The notification approach was adapted for DIBA Bookings. Notifications are linked to individual users and can also be linked to a booking. Users can retrieve their own notifications, view whether a notification has been read and mark unread notifications as read.

1.3.5 Audit Logging

Description

The Accessly project was reviewed to understand how important system actions can be recorded in an audit log. The project records actions together with information about the actor and affected entity (Lakhili, 2026).

The Headless Framework project was also reviewed because it includes audit logging as one of its backend features (Shaheen, 2026).

Usage in DIBA Bookings

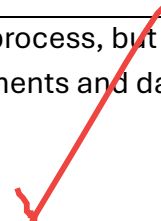
The audit logging approach was adapted for DIBA Bookings. The system records important actions such as booking approval, booking rejection and booking cancellation. The audit record stores information such as the action, description, date and user who performed the action. This allows administrators to review important activities in the system.

1.3.6 Summary of Adapted External Sources

The external sources were not used as complete solutions for DIBA Bookings. They were used to understand specific technical problems and possible ways of solving them. The approaches were then changed to fit the DIBA system.

External concept	DIBA Bookings adaptation
JWT and role-based authorisation	Adapted for Administrator, Conference Centre Staff and Event Organiser roles.
Booking conflict detection	Adapted to check venue, date/time overlap and booking status.
Conference booking management	Adapted to support DIBA's booking, approval and rejection process.
User notifications	Adapted to provide user-specific booking notifications and read/unread status.
Audit logging	Adapted to record important booking and administrative actions.

The external sources therefore supported the development process, but the final implementation was structured around the specific requirements and database design of the DIBA Bookings system.



1.4 Problems Encountered

Several problems were encountered during the development of the DIBA Bookings system. The problems included technical difficulties, limited development time, access to computers, team coordination and the amount of work required to develop the system from the beginning.

1.4.1 Developing the Project Idea and Requirements

One of the first challenges was that the team was not given an existing application to improve or a complete system specification to follow. The DIBA Bookings idea was developed by the team and then proposed to the lecturer.

This meant that the team had to decide what problem the system would solve, what features it should have, who would use it and how the booking process should work. The team had to identify the different users, including Event Organisers, Conference Centre Staff and System Administrators. The team also had to decide what each user should be allowed to do.

This required the team to spend time discussing the idea, researching possible solutions and changing some parts of the system before development could begin. The problem was managed by breaking the project into smaller requirements and documenting the system before implementing the main features.

1.4.2 Designing the System Before Development

Another challenge was creating the technical design of the system. The team had to create the Software Requirements Specification (SRS) and design the structure of the application before implementing it. This included deciding what information the system needed, how the different parts of the system would connect and how users would interact with the system.

Creating the class diagram in Visual Studio was also part of this process. The team had to identify the main classes, their properties and the relationships between them. For example, the team had to determine how Users, Roles, Events, Venues and Bookings should be related.

This was difficult because changes made to the design could also affect the database, backend and frontend. The problem was managed by reviewing the requirements and gradually improving the system design before implementing the related features.

1.4.3 Creating the Database Structure

After designing the system, the team had to create the database structure required by the application. The team understood the information that needed to be stored, but it was initially difficult to understand how the C# entity classes, Entity Framework Core, the DbContext and SQL Server worked together.

The team had to ask questions such as:

- which tables were required?
- which fields were needed?
- which fields should be primary keys?
- which fields should be foreign keys?
- how tables should be related? and
- how changes to one part of the system could affect related records?

The problem was addressed by studying Entity Framework Core documentation, reviewing the class design and testing the database relationships during development. The team gradually corrected the entity relationships and database configuration until the backend could successfully communicate with the database.

1.4.4 Complexity of the Bookings Controller

The booking functionality became one of the most complicated parts of the system. At the beginning, creating a booking appeared to be a simple process. However, as the requirements were implemented, the booking process required many checks and actions.

The controller eventually had to handle:

- creating bookings;
- checking dates;
- checking that venues exist;
- checking venue availability;
- checking that events belong to the correct user;
- detecting conflicting bookings;
- updating bookings;
- approving bookings;
- rejecting bookings;
- cancelling bookings;
- changing booking statuses;
- creating notifications; and
- creating audit records.

This caused the BookingsController to become much larger and more complex than initially expected.

The team managed this by separating the different operations into different API endpoints and using DTOs and Entity Framework Core queries to keep each operation focused.

1.4.5 Authentication and Authorisation

Implementing authentication and authorisation was another challenge. The system needed to identify a user after login and then determine what that user was allowed to access.

Several parts had to work together. The user had to log in, the backend had to check the login details, a JWT had to be created, and the frontend had to send the token when accessing protected endpoints. The backend then had to read the token and identify the user's role before allowing or denying access. Understanding how these parts worked together was initially difficult.

The problem was addressed by testing the authentication endpoints using Swagger before connecting them to the React frontend. Different user roles were also tested to check that users could only access functions allowed for their role.

1.4.6 Booking Availability and Conflicting Bookings

Another technical challenge was preventing two users from booking the same venue at overlapping times. The system could not simply check whether a venue already had a booking. It had to compare the requested start and end times with existing bookings. For example, if a venue was booked from 10:00 to 12:00, another booking from 11:00 to 13:00 should not be allowed.

The team therefore had to understand how date and time ranges overlap and then apply this logic to the DIBA Bookings requirements. The final implementation checks the venue, the requested time period and the status of existing bookings before creating a new booking.

1.4.7 Managing Booking Statuses

The booking process also became more complicated because bookings could have different statuses.

The system uses statuses such as:

- Pending;
- Approved;
- Rejected;
- Cancelled; and
- Completed.

Not every action should be available for every status. For example, a pending booking can be approved or rejected, while an already rejected booking should not be approved. The team therefore had to add checks before allowing status changes. This required the team to think about the booking process as a sequence of possible states rather than treating every booking in the same way.

1.4.8 Frontend and Backend Communication

Connecting the React frontend to the ASP.NET Core backend was another challenge. The frontend had to send requests to the correct API endpoints and provide the information expected by the backend. The backend also had to return information in a format that the frontend could use.

A problem on either side could cause a feature to fail even when the other side was working correctly. The team addressed this by testing the backend endpoints separately using Swagger before connecting them to the React frontend. This made it easier to identify whether a problem was coming from the frontend or backend.

1.4.9 Testing Role-Specific Functionality

Testing was challenging because the system has different user roles with different permissions. An Event Organiser, Conference Centre Staff member and System Administrator should not have access to exactly the same functions.

The team therefore had to create and test different user accounts and check both allowed and restricted actions. This helped identify authorisation problems and confirmed that role-based access was being applied correctly.

1.4.10 Coordinating Work Between Team Member

The team's different class schedules also affected development. There were times when one team member was in class while the other was available to work on the project. This made it difficult to always work on the same part of the system at the same time.

The team managed this by communicating about completed work, sharing changes through GitHub and continuing with separate tasks when both members were not available at the same time. This allowed development to continue even when the team members were not physically working together.

1.4.11 Limited Development Time and Laboratory Access

Limited access to university computers was another major challenge. The team was not given enough time to use the laboratory computers for all the work required by the project. The laboratories also had limited operating hours, which reduced the amount of time available for development.

This was especially difficult because software development often requires time to install tools, open the development environment, build the application, test the system and investigate errors.

There were situations where the team was still working on a problem when the available laboratory time was ending. The team therefore had to make careful use of the available

laboratory time and continue working outside the laboratories when possible. This limitation affected the speed of development and required the team to manage its time carefully.

1.4.12 Slow Development Computers

The performance of our computers also affected development. Opening Visual Studio, building the backend, running SQL Server and testing the application could take a significant amount of time on slower computers.

This increased the time needed to test changes and investigate problems. The team managed this by planning tasks carefully and using available development time as efficiently as possible.

1.4.13 Managing the Project Timeline

Because the team had to develop the project from the idea stage, there were many deliverables that had to be completed before the final implementation. The team therefore created a project outline using Microsoft Excel to schedule the deliverables and planned when different parts of the project should be completed.

However, technical problems sometimes took longer to solve than expected. This caused some planned tasks to take longer and affected later tasks. The team responded by changing priorities when necessary and focusing on important tasks first.

1.4.15 Map and Location Functionality

Location functionality also required additional consideration. The system was designed to provide users with information about venue locations. The team therefore had to decide how location information should be presented while keeping the feature within the scope of the project.

The team focused on providing useful venue location information without making the location feature unnecessarily complicated.

1.5 Implementation Summary

The implementation phase changed the DIBA Bookings idea developed by the team into a working web application. The team was responsible for developing the idea, proposing it to the lecturer, identifying the system requirements, designing the system and implementing the final application. The team was not given an existing application or complete system to build from.

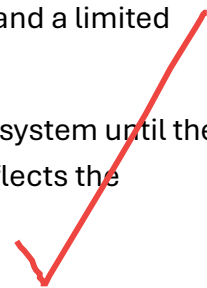
The final system consists of a React frontend, an ASP.NET Core Web API backend and a Microsoft SQL Server database.

The main implemented features include:

- user registration and authentication;
- role-based access;
- venue management;
- event management;
- booking creation;
- booking availability checking;
- booking approval and rejection;
- booking cancellation;
- notifications;
- payment records;
- audit logging; and
- administrative functionality.

The implementation process involved both technical and practical challenges. The team had to learn how the different technologies worked together, create the system design and database structure, implement authentication and authorisation, develop booking rules and connect the frontend to the backend. The team also had to work with limited laboratory access, slow computers, different class schedules and a limited project timeline.

Despite these challenges, the team continued to develop and test the system until the main planned functionality was implemented. The final application reflects the requirements and design developed by the team during the project.



References

Hisham, O. (2026). *Omar Hisham*. Retrieved from gihub:
<https://github.com/Oh011/NotificationService.git>

Iriarte, J. M. (2026). *Microservices Infrastructure Personal Finance Dashboard*. Retrieved from github: <https://github.com/tigerbluejay/Microservices-Infrastructure-Personal-Finance-Dashboard.git>

JedAngelo. (2024). *ConferenceBookingAPI*. Retrieved from gihub:
<https://github.com/JedAngelo/ConferenceBookingApi.git>

Lakhili, M. (2026). *Accessly*. Retrieved from github:
<https://github.com/LMouhssine/Accessly.git>

Microsoft. (2026). *Microsoft Ignite*. Retrieved from Microsoft Learn:
https://learn.microsoft.com/en-us/aspnet/core/security/authorization/roles?view=aspnetcore-10.0&utm_source=chatgpt.com

olaideogunbunmi. (2026). Retrieved from Github:

<https://github.com/olaideogunbunmi/aspnetcore-auth-rbac.git>

ouardini14. (2022). *Meeting-room-reservation-api*. Retrieved from github:

https://github.com/ouardini14/Meeting_Planner.git

Shaheen. (2026). *Headless Framework*. Retrieved from github:

<https://github.com/xshaheen/headless-framework.git>

Index of comments

- 1.1 Remove the draw reader with an engaging abstract. It is typically a short summary of the document
- 4.1 You can guide the reader to know to which sections to expect i.e Section 1.1 provides choice of tools, while Section 1.2 presents code extracts ..
- 19.1 Add references on the next separate page